

POLYTECHNIQUE MONTRÉAL

affiliée à l'Université de Montréal

**Digital Twin for Monitoring of the Bus Fleet of Société de Transport
de Montréal**

by

Kérian FITER (2402537)

Vanny KATABARWA (2349909)

Carlos PAMBO (2346917)

LOG6953FE - Digital Twin Engineering Project Report

April 2025

Contents

List of Figures	ii
1 Introduction	1
1.1 Background	1
1.2 Objectives	1
1.3 Assumptions and Limitations	2
1.4 Report Organization	2
2 Architecture of the System	3
2.1 Architecture of the Digital Twin	3
2.1.1 Digital Twin, Shadow, or Model?	4
2.1.2 Data Description	4
2.2 Characterization of the Digital Twin	5
2.3 Summary	7
3 Services of the System	8
3.1 Route Visualization	8
3.1.1 3D Map Creation	8
3.1.2 From GPS Coordinates to Position on the 3D Map	9
3.1.3 Visualizing the Bus Fleet	10
3.2 Telemetry Validation Service	11
3.3 Notification Service	13
3.4 Summary	15
4 Conclusion	16
4.1 Challenges	16
4.2 Summary of Project	16
4.3 Suggestions for Future Work	17
References	18
A Acronyms and Notations	19
B Resources	20

List of Figures

2.1	STM DT architecture	3
3.1	Montréal model (textured) in Blender	8
3.2	Map tiles of various sizes in Blender	9
3.3	Reference markers on the map in Godot	10
3.4	Bus asset in Godot.	10
3.5	Bus and ghost bus transition diagram	11
3.6	Email notification sample	14

1. Introduction

This progress report outlines the current status of the “**Digital Twin for Monitoring of the Bus Fleet of Société de Transport de Montréal**” project. The project aims to build a digital model representing the real-time status of STM bus routes to enhance operational efficiency, reduce downtime, and improve the passenger experience. This report presents the DT, highlighting the services built, the technology used, and the characteristics of the DT.

1.1 Background

The Société de transport de Montréal (STM) manages a large and vital bus fleet, which is essential for public transportation in Montréal, Québec, Canada. To improve the operational efficiency of its bus fleet and provide a more reliable service, we propose building a digital twin (DT) for its bus fleet.

In [3], the authors defined a DT as a digital representation of a physical asset. In this instance, a digital representation of the STM bus fleet is used. To improve the real-time operational performance of the fleet. The DT use data and analytics to identify unusual bus conditions before they become serious problems. The DT can have services that will send alerts to the STM operation center regarding such issues, or such services can notify passengers in near real-time so they won't have to wait at the bus stops wondering when the bus will arrive. This proactive approach ensures both smoother operations and a better experience for passengers.

Public transportation is essential for efficient movement in cities like Montréal. Our team is developing a DT for the STM bus lines to address the need for real-time insights. This project will create a virtual model that reflects live bus movements, enabling better monitoring and faster responses to routing issues, ultimately enhancing the commuter experience.

By integrating real-time data and global positioning system (GPS) telemetry anomaly detection, the DT empowers STM operators to make informed decisions. This initiative meets technical objectives and highlights the potential of DTs in modern transportation systems.

1.2 Objectives

This project aims to:

- I To *develop* a virtual representation of the STM bus lines telemetry data. Reflecting the (near) real-time position data of the STM buses.
- II To *develop* a visualization tool for the bus lines.
- III To *develop* a GPS telemetry validation service.
- IV To *develop* an alert system to notify STM teams when potential bus telemetry issues are detected.
- V To *recommend* suggestions for future work that follows the work reported in this

1.3 Assumptions and Limitations

The project is developed under the following assumptions and limitations:

- i STM provides authenticated open access to bus telemetry data through an application programming interface (API).
- ii STM does not restrict access and use of the telemetry data of the buses.

1.4 Report Organization

Following this introductory chapter, the remainder of this report comprises three more chapters, one appendix, and a bibliography.

Chapter 2 describes the adopted architecture of the DT. **Chapter 3** discusses the services built on for the DT. The chapter also discusses the telemetry validation algorithm. The correct chapter, in conjunction with Chapter 2, addresses Objectives II, III, and IV. **Chapter 4** concludes this report by assessing the work done and recommendations for future work, fulfilling Objective V of the report.

2. Architecture of the System

2.1 Architecture of the Digital Twin

This section of the report discusses the architecture of the STM DT. Figure 2.1 depicts the architecture of the DT. The DT system ingests real-time data (GPS coordinates, bearing, speed) from the STM bus fleet through an application programming interface (API). The data flows from the STM API into the DT, through a set of services described as “Enablers” in the architecture diagram. These “Enablers” include RabbitMQ for messaging, MongoDB for storage, and an ingestion pipeline built on top of the DT for data ingestion, that fetches the data from the STM API, pre-processes and distributes to all subscribing services.

A suite of “Services” was built as part of the DT. These include a visualization service, a GPS telemetry validation service with deviation computation, and a notification service. Each of these services processes the data using a Pub/Sub architecture ¹.

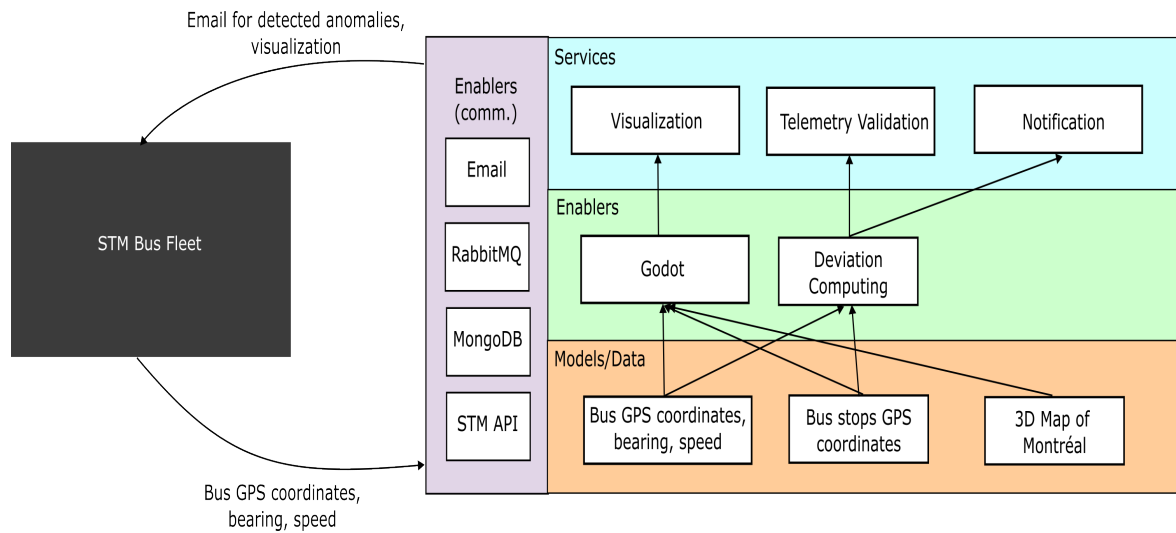


Figure 2.1: STM DT architecture

The following services are built as part of the project:

- i) **Route visualization:** displays (near) real-time bus locations;

¹The Publish/Subscribe architecture is a messaging pattern where senders (publishers) broadcast messages to a channel (topic). Subscribers receive only the messages they are interested in, based on the topics they subscribe to

ii) **Telemetry validation:** Examines incoming bus telemetry data (speed, location, bearing) to spot unexpected deviations or faults;

iii) **Notifications:** dispatches email notifications with anomalous data to define lists of recipients;

They use the following enablers:

i) Deviation computing: computes how far actual bus movements deviate from a planned schedule or route using the GPS telemetry data from the STM API;

ii) Godot: Fetches pre-processed data and displays (near) real-time bus and stop positions on a 3D map of Montréal;

The following data-exchange services are built as part of the project:

i) RabbitMQ: routes external (STM) and internal (other DT services) data events;

ii) MongoDB: stores and retrieves historical data for analytics, including the location of bus stops for each of the buses;

iii) Emails: When anomalies are detected, a notification is sent to the STM operations center via email;

2.1.1 Digital Twin, Shadow, or Model?

The DT is continuously updated from (near) real-time bus telemetry data and provides insights and notifications to STM. However, it does not directly send control signals back to the fleet. Instead, our DT architecture operates with a “man-in-the-middle” approach. Since emails rather than direct control signals to the buses are sent out from DT, there is no automated feedback loop to change the fleet operation. Consequently, the team agrees that our architecture aligns more closely with that of “**digital shadow**” definition. Although our digital system is a live, automatically updated digital representation of the physical world with analytics and alerting capabilities, the digital services do not close the loop by actively controlling or adjusting its physical twin counterpart, the buses in our digital twin.

2.1.2 Data Description

The STM API [5] returns data in Protobuf, short for Protocol Buffers ¹. The DT data ingestion service converts the DT data format into Java object notation (JSON) format for

¹Protobuf is a language and platform-neutral, and extensible mechanism for serializing structured data, developed by Google, for efficient exchange of data

2. ARCHITECTURE OF THE SYSTEM

2.2 Characterization of the Digital Twin

further processing. Table 2.1 summarizes the name of the fields, format and a short description of the data returned from the API. A complete description of the data dictionary for the STM API can be found in [2].

Field	Type	Description
id	String	A unique identifier for the trip
vehicle	JSON	Encapsulates the information of the bus
vehicle.trip	JSON	Encapsulates the information of the trip
vehicle.position	JSON	Encapsulates the information of the bus position
vehicle.current_stop_sequence	Integer	Current stop number in the line
vehicle.current_status	Integer	The exact status of the vehicle with respect to the next bus stop on the line
vehicle.timestamp	Integer	Unix timestamps in seconds of when the record was captured at the source (bus)
vehicle.occupancy_status	Integer	The state of passenger occupancy for the bus
vehicle.trip.trip_id	String	A unique identifier for the trip
vehicle.trip.start_time	String	The time the trip started
vehicle.trip.start_date	String	The date of the trip
vehicle.trip.route_id	String	Bus line number
vehicle.position.latitude	Float	Bus GPS latitude
vehicle.position.longitude	Float	Bus GPS longitude
vehicle.position.bearing	Float	Bus GPS bearing
vehicle.position.speed	Float	The speed the bus is traveling at the moment the reading is captured

Table 2.1: DT data payload description

2.2 Characterization of the Digital Twin

Based on the DT reporting characteristics in [1], the attributes of our DT are summarized in Table 2.2. The code and the instructions to run our DT are available at the GitHub repository [4].

	Characteristic	Our DT
C_1	System under study	STM bus fleet
C_2	Physical acting components	Buses telemetry (no direct control from the DT)
C_3	Physical sensing components	GPS telemetry sensors on the buses
C_4	Physical-to-virtual interaction	Real-time GPS telemetry data ingestion via an API
C_5	Virtual-to-physical interaction	None (no feedback control, hence, the system is classified as digital shadow)
C_6	DT services	Bus route visualization; GPS telemetry validation; Notification and alerts (emails)
C_7	Twinning time-scale	Near real-time; Updated via GPS telemetry data streaming
C_8	Multiplicities	Digital shadow of the STM bus fleet
C_9	Life-cycle stages	Operational monitoring phase
C_{10}	DT models and data	JSON GPS telemetry data; and, GTFS-based trip data
C_{11}	Tooling and enablers	Python, C#, RabbitMQ, Godot, MongoDB, Blender, SMTP
C_{12}	DT constellation	Standalone DT (not part of broader system constellation)
C_{13}	Twinning process and DT evolution	DT ingests evolving (near) real-time data, but no auto-updates to physical system
C_{14}	Fidelity and validity considerations	GPS telemetry triangulation and validation algorithm for fidelity checks
C_{15}	DT technical connection	API (STM-to-DT), RabbitMQ (internal), SMTP (DT-to-STM)
C_{16}	DT hosting/deployment	Deployed locally using Python services, local instances of RabbitMQ, MongoDB, and Godot for visualization of the routes
C_{17}	Insights and decision making	Man in the middle decision approach, with STM notified of anomalies in the fleet
C_{18}	Horizontal integration	Partial, uses multiple tool chains but limited external system integration
C_{19}	Data ownership and privacy	Depends on STM's open data access policy (assumed unrestricted)
C_{20}	Standardization	GTFS standards for telemetry data
C_{21}	Security and safety considerations	No explicit mechanisms reported; assumes demonstration-level security

Table 2.2: Summary of DT characteristics

2.3 Summary

The chapter described the proposed architecture of the DT, including a description of the data ingested by the DT, a discussion of the services built, and the characterization of the DT regarding the twining, shadow or model paradigm. The digital system was characterized as a digital shadow because of the one-way data flow from the physical buses to the digital service. Lastly, the chapter moved to describe the DT about the reporting framework [1]. The next chapter discusses in more detail the services built for the DT.

3. Services of the System

3.1 Route Visualization

Visualizing the bus fleet is essential to get an immediate sense of the PT. Having it in real-time is a critical characteristic that makes it worthwhile. We chose to do a 3D visualization rather than a classic 2D visualization because it was a more interesting take, providing *fancy visuals*, and a real sense of where the buses are in the city.

3.1.1 3D Map Creation

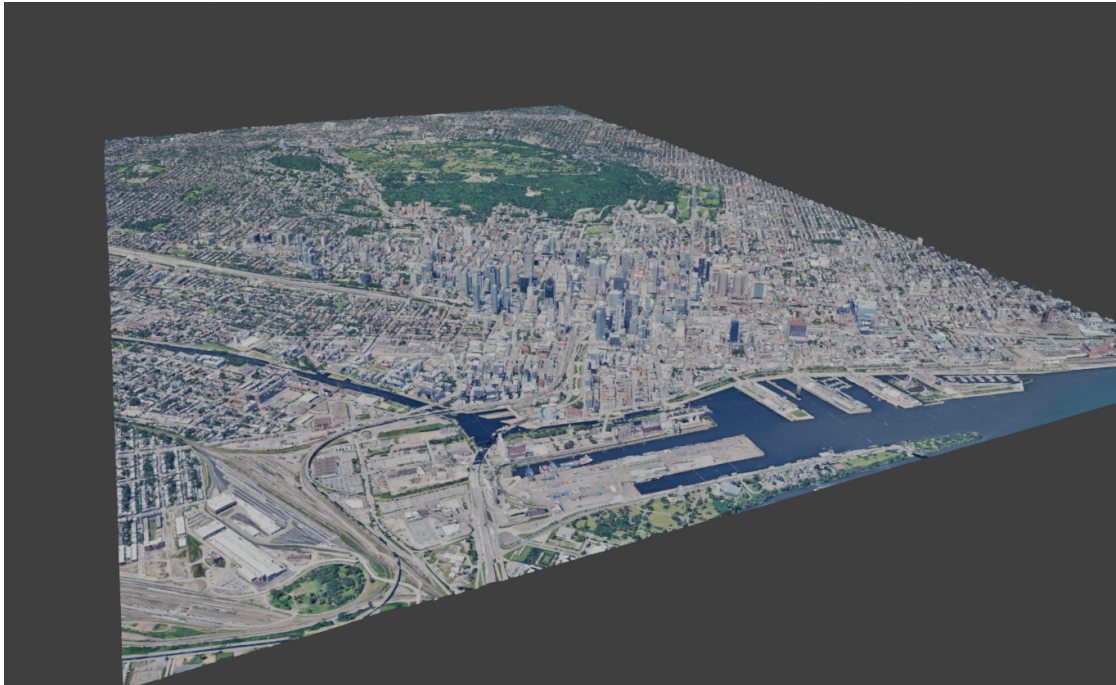


Figure 3.1: Montréal model (textured) in Blender

We used a plugin called Blosm that allowed us to generate a 3D map of Montréal in Blender from the Google Maps Tile API based on GPS coordinates. This process had to be done iteratively and with a reduced level of detail due to the resource-intensive nature of the process. We also had to remove textures due to performance issues. We also kept the default API's setting of separate tiles, rather than having one big merged map object. It was more efficient in both Blender and Godot.

Blosm has a few parameters that we experimented with to obtain a compromise between fidelity and performance, as you can see in Figure 3.2, not all tiles are the same size. This is because we chose to generate tiles of lower precision for Mont-Royal, which had few roads but many detailed trees. This allowed us to improve performance and reduce the final map size.

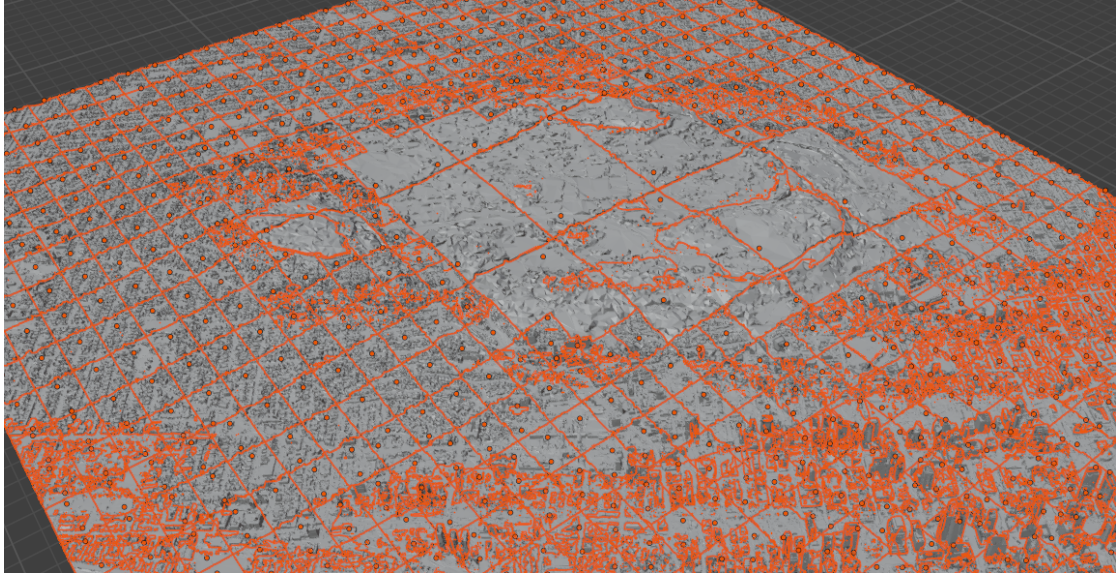


Figure 3.2: Map tiles of various sizes in Blender

The map was then exported in GLTF format. Its size is about 600 Mb, which is still manageable. Then it was imported into Godot.

3.1.2 From GPS Coordinates to Position on the 3D Map

Once the map is imported into Godot (Mono version, 4.4), the GPS positions we receive from the STM API must be mapped to the 3D map. We chose to do triangulation based on the closest reference points to avoid any drift. First, we select landmarks close to the bus routes and easily recognizable on a 3D map. We then put anchors in the 3D map containing fields for their accurate latitude and longitude (see Figure 3.3). Now, when we receive a new latitude and longitude, we can determine which three closest reference points are, and use triangulation to find the estimated position on the map.

This method has benefits and drawbacks:

- No need to determine the right scaling ratio between, triangulating does it automatically.

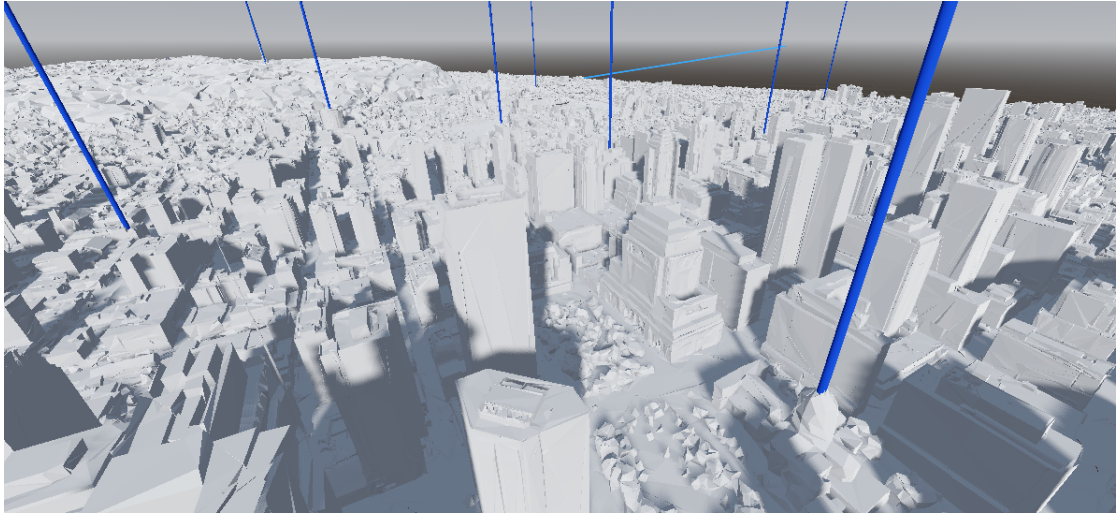


Figure 3.3: Reference markers on the map in Godot

- Time-consuming to place all the reference markers by hand.
- Difficult to precisely place markers on the 3D map without distinct landmarks.

3.1.3 Visualizing the Bus Fleet

We first had to obtain RabbitMQ bus fleet information from Godot in real time. We used the official RabbitMQ C# integration with Godot. We then parse the incoming data into JSON and spawn buses not yet present in the scene. This is easy because each bus is assigned a unique ID, which we use to check if the bus exists.

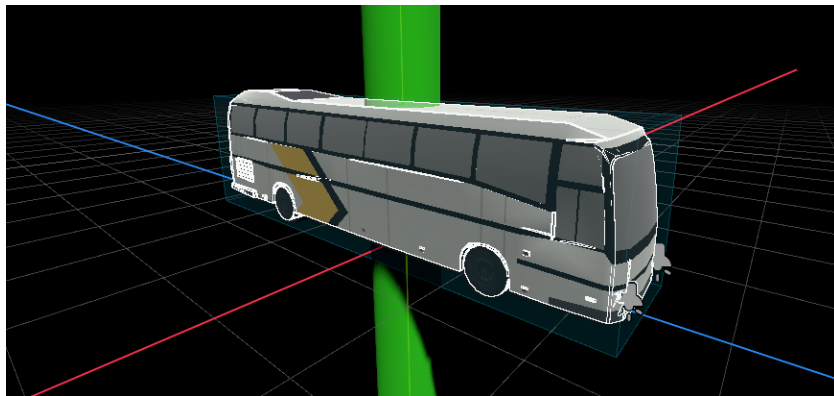


Figure 3.4: Bus asset in Godot. “bus” (<https://skfb.ly/6wprD>) by The_Ping Pro is licensed under Creative Commons Attribution (<http://creativecommons.org/licenses/by/4.0/>).

We then spawn a bus at the correct location. We also added a vertical marker and an outline so you can see it from afar easily. We implemented the ability to click on the bus, lock onto it, and follow it.

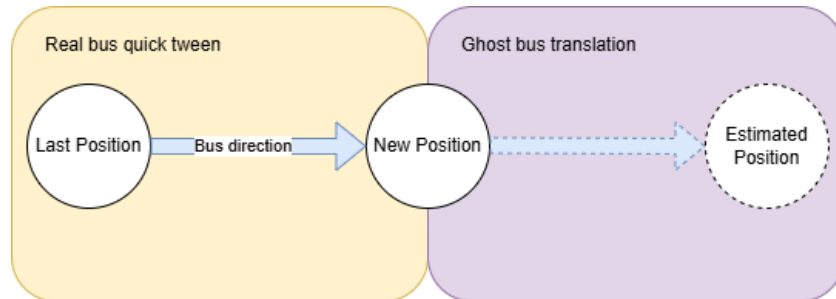


Figure 3.5: Bus and ghost bus transition diagram

We added an interpolation (using Tween) between the last and newly received positions to make the bus fleet feel more realistic. The delta between those two positions gives us a 2D vector, which we use to estimate the next position by adding it to the new position. We then interpolate a semi-transparent and non-textured bus version to this estimated position. It gives the user a feeling of both the speed of the bus and its direction. This behavior is laid out in Figure 3.5.

3.2 Telemetry Validation Service

This service uses a model-free algorithm to classify GPS telemetry data from the buses. Algorithm 3.1 provides a pseudo-code description of the process used to classify anomalous telemetry data. The algorithm inputs and outputs the following:

- **Inputs:**
 - The bus line number, represented as **B**.
 - A list of GPS telemetry data, in the format of the five-element tuple, (latitude, longitude, bearing, speed, stop sequence), represented as **C**.
- **Outputs** (classifies the telemetry reading into one of three classes) :
 - **OK**, if the coordinates are valid;
 - **NOT_FOUND**, if no bus line (route) information entry is found in the MongoDB database;
 - **ANOMALY**, if the coordinates are classified as anomalous

Algorithm 3.1: GPS telemetry validation algorithm**Input** : Bus line **B**, List of GPS coordinates (**C**)**Output**: List of validated GPS coordinates (**C'**)

```

1 Initialize C' with C;
2 for  $i = 1$  to  $C_{size}$  do
3   Select coordinates in  $C_i$  and save it in X;
4   Ensure all required fields exist in X ;
5   Query the database to get bus stop information for the bus line B and save it in R ;
6   if  $bus\_stops \in R$  then
7     Initialize THRESHOLD with Maximum_Distance value in R ;
8     Determine the coordinates status by comparing positions ;
9     Extract bus stops from R, or use an empty list if not present and store it in T ;
10    Get the maximum allowed distance from T and store it in M ;
11    if  $T \neq \emptyset$  then
12      Sort the routes in T ;
13      Get the bus current stop sequence from  $C_i$  and store it in B ;
14      Calculate the distance between the bus stop and the bus using latitude and
        longitude coordinates and store it in D ;
15      if  $D \leq THRESHOLD$  then
16        Update C' coordinate status at index  $i$  to OK ;
17         $i++$  ;
18      if  $THRESHOLD \leq D \leq MAX\_ALLOWED\_DISTANCE$  then
19        Use the bus stop in T for a new distance calculation and store it in D;
20      if  $D > MAX\_ALLOWED\_DISTANCE$  then
21        Reverse the bus stop order in T and recalculate for a distance calculation
          and store it in D;
22      if  $D \leq THRESHOLD$  then
23        Update C' coordinate status at index  $i$  to OK ;
24      else
25        Update C' coordinate status at index  $i$  to ANOMALY ;
26         $i++$  ;
27      else
28        Update C' coordinate status at index  $i$  to NOT_FOUND ;
29         $i++$  ;
30    else
31      Update C' coordinate status at index  $i$  to NOT_FOUND ;
32       $i++$  ;
33 return C'

```

We decided to follow this model-free approach to classify GPS telemetry data because, to our knowledge, there is no publicly available dataset for this. Since the team could not create a labeled dataset of the data required to train a machine learning (ML) model, we decided to develop an algorithm 3.1 instead.

3.3 Notification Service

The notification service is the third independent component/service built as part of the DT that gets triggered when the telemetry validation service detects an anomaly. When the computed distances between the reported position of the bus and the expected bus stop coordinates exceed the allowed threshold, such telemetry coordinates are categorized as an **ANOMALY**. An event message is published to the specified RabbitMQ topic to which the notification service subscribes.

Using this event-driven approach helps decouple the validation logic from the notification process, thus ensuring:

- a) The telemetry anomaly service can continue processing incoming data without delay.
- b) The notification service can handle spikes in notifications (if multiple anomalies are detected) without impacting the telemetry validation processes.

Upon receiving the event message from the telemetry anomaly service, the notification service constructs an email message that includes the following information:

- Bus information like current stop sequence, GPS coordinates, etc.
- Trip information like trip_id, etc.
- A timestamp of the event.

Figure 3.6 shows an example of the email dispatched by the service. The service uses simple mail transfer protocol (SMTP) to send the email to a pre-configured list of recipients; personal emails were used for demo purposes. The email content is formatted to highlight the anomaly and prompt immediate attention.

Société de Transport de Montréal Bus Fleet Digital Twin

Good day Operations Team,

This email outlines the need for immediate corrective actions regarding telemetry data for the following bus lines:
121.

Here is a summary of the issues that required **immediate** actions:

Trip-Id	Line	Timestamp	Bus-Stop	Status	Occupancy
284300929	121	2025-03-29 16:39:39	Station Angrignon	INCOMING_AT	EMPTY
284300945	121	2025-03-29 16:39:38	Station Angrignon	STOPPED_AT	MANY_SEATS_AVAILABLE
284301058	121	2025-03-29 16:39:38	Station Angrignon	INCOMING_AT	EMPTY
284300693	121	2025-03-29 16:39:37	Station Angrignon	INCOMING_AT	EMPTY
284301016	121	2025-03-29 16:39:35	Station Angrignon	STOPPED_AT	MANY_SEATS_AVAILABLE

Please prioritize taking actions to remediate issue(s), if need be confirm the readings of the bus driver.

Regards

Copyright © 2025 | LOG6953FE - Digital Twin Engineering

Figure 3.6: Email notification sample

3.4 Summary

The chapter described in detail three of the services built for the project. The visualization service is the component that graphically represents the bus lines, stops, and real-time positions. The chapter then discusses the telemetry validation service, which continuously validates real-time bus location data accuracy against expected bus stop coordinates. The service acts as a watchdog by calculating distances and determining anomalies, ensuring that deviations from the planned route are detected promptly. The last service discussed is the notification service, which guarantees that the right teams receive timely alerts, enabling a rapid operational response when discrepancies occur. The next chapter presents a conclusion of the report, highlighting some of the challenges and suggestions for future directions to improve the DT.

4. Conclusion

4.1 Challenges

Below is a breakdown of the challenges the team has faced so far in building the project:

- i) STM bus data contained missing entries or inconsistencies, requiring significant pre-processing efforts. One specific instance is the omission of `direction_id` property from the `Trip` object, requiring extra processing to determine the direction of the bus. This information is needed in all subsequent services of the DT.
- ii) Developing an optimized data retrieval mechanism to ensure efficient real-time monitoring due to STM implementation of rate limiting on its API.
- iii) The 3D map of Montréal was originally too resource-heavy and has been optimized by stripping textures and reducing vertices. Mapping coordinates to the 3D map was imprecise, with only one reference point. Triangulation based on three closest reference points has been implemented to minimize imprecision.

4.2 Summary of Project

In **Chapter 1**, the DT for the STM bus fleet monitoring project is introduced, and the aims of the project are described. The chapter then outlines the primary objectives of the project, which include creating a digital representation of the telemetry data of the buses, a visualization tool for monitoring the buses, a GPS telemetry validation service, and an alert notification service. The chapter also details the assumptions and limitations regarding access to STM telemetry data. The chapter concludes by summarizing the organization of the report.

Chapter 2 outlines the architecture of the DT for the STM bus fleet. The chapter details how real-time bus telemetry data from the STM API is ingested and processed through various DT services. The chapter then argues that the system operates as a digital shadow due to the lack of control signals flowing from the digital representation to the physical asset being twined. The chapter then comprehensively characterizes the DT based on its established attributes. The chapter summarizes the DT design and operational framework, setting the stage for a deeper exploration of the built services in the subsequent chapter.

Chapter 3 details the core services of the DT. It starts with the route visualization service that leverages a 3D map of Montréal. It integrates GPS coordinates from the STM buses to

display real-time bus positions, explaining the process of mapping GPS coordinates onto the 3D map via triangulation with manually placed reference markers and animating them within the environment. The chapter then describes the GPS telemetry validation service, which uses a model-free algorithm to classify incoming GPS data into valid or anomalous categories by comparing bus positions with expected bus stop coordinates. The chapter then describes the notification service dispatching email alerts through SMTP when anomalies are detected, ensuring that the STM operations team receives prompt notifications for deviations.

Chapter 4 reviews the challenges, accomplishments, and future directions of the project.

4.3 Suggestions for Future Work

This project investigated the feasibility of developing a DT for the STM bus lines. The project showed that it is feasible to build such a DT, but as in every project, there are still opportunities for improvements. Below are some of the opportunities:

- I) Perform telemetry validation using ML algorithms instead of the current model-free approach.
- II) Expand the current digital representation from a digital shadow to a DT by allowing the system to send control signals to its physical twin. This would enable digital representation to detect anomalies, adjust bus operations, or reroute vehicles automatically in (near) real-time.
- III) Incorporate additional data sources such as weather conditions, traffic data, and passenger load information to provide a more comprehensive system view.
- IV) Expand the notification system to include multi-channel alerts (SMS, mobile push notifications, etc.) and integrate escalation protocols for critical situations, ensuring timely responses.

References

- [1] GIL, S., OAKES, B.J., GOMES, C., FRASHERI, M. & LARSEN, P.G. (2025). Towards a Systematic Reporting Framework for Digital Twins: A Cooperative Robotics Case Study. *Simulation*, **101**, 313–339. 5, 7
- [2] GOOGLE (2022). Google Transit GTFS Schedule Reference and Differences — Static Transit. <https://developers.google.com/transit/gtfs/reference>, [Accessed 01-03-2025]. 5
- [3] OAKES, B., GOMES, C., KAMBURJAN, E., ABBIATI, G., ECEM BAS, E. & ENGELSGAARD, S. (2024). Towards Ontological Service-Driven Engineering of Digital Twins. In *Proceedings of the ACM/IEEE 27th International Conference on Model Driven Engineering Languages and Systems*, 464–469. 1
- [4] PAMBO, C., FITER, K. & KATABARWA, V. (2025). Société de Transport de Montréal Bus Digital Twin. https://github.com/carlospambo/Society_de_Transport_Montreal_DT, [Accessed 14-04-2025]. 5
- [5] SOCIÉTÉ DE TRANSPORT DE MONTRÉAL (2025). STM Developer Portal. <https://api.stm.info/pub/od/gtfs-rt/ic/v2/vehiclePositions>, [Accessed 01-03-2025]. 4

A. Acronyms and Notations

The acronyms are listed in alphabetical order.

2D	Two-Dimensional
3D	Three-Dimensional
API	Application Programming Interface
DT	Digital Twin
GPS	Global Positioning System
GTFS	General Transit Feed Specification
JSON	Javascript Object Notation
ML	Machine Learning
PT	Physical Twin
SMTP	Simple Mail Transfer Protocol
STM	Société de Transport de Montréal

B. Resources

This appendix contains the URLs to the GitHub repository and the YouTube link for the DT explainer video summarized in Table B.1.

Resource	URL
GitHub	https://github.com/carlospambo/Society_de_Transport_Montreal_DT
YouTube	https://www.youtube.com/watch?v=ezhNTBHXkss

Table B.1: Summary of DT Resource URLs